Project Proposal Presentation

- Upload to Assignments > Term Project > Term Project Proposal
  - 5% of your project grade
    - Teams are required to present a 5-10 minute proposal to introduce their project to the class. The presentation should be in PPT or PDF format and include at least 5 slides covering the following:
      - **Team Name and Project Overview:** Briefly introduce the team and outline the project objectives.
      - **AWS Services Overview**: List the AWS services you plan to use, with a brief description of how each will support project goals.
      - **High-Level Data Flow Diagram**: Present a diagram showing the interaction of AWS services and the primary data flow within the project.
  - Check out https://app.diagrams.net for drawing diagrams
    - It's free and offers lots of AWS shapes
  - More details can be found at Content > Project > 514 Term Project

RECAP

- In Kubernetes, what is the smallest deployable unit?
  A. Node
  B. Service
  C. Pod ✅
  D. Cluster

- In Kubernetes, a namespace is used to group containers within a pod
  - True
  - False ✅

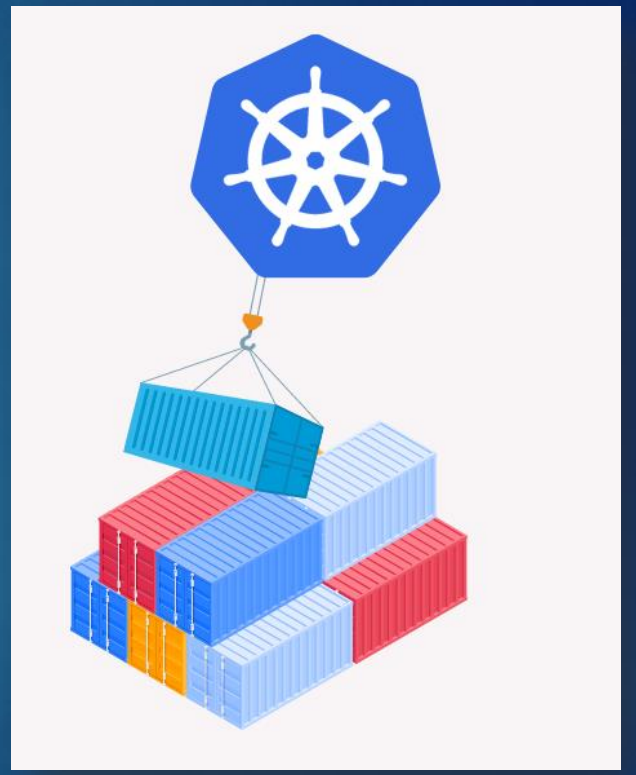
- Amazon ECS supports both EC2 and Fargate launch types
  - True ✅
  - False

RECAP

- Container orchestration automates the deployment, scaling, and management of containerized applications across a distributed system
- Key features include:
  - **Provisioning and Deployment**: Automates container creation and placement across servers
  - **Scaling**: Dynamically adjusts resources (up or down) based on workload demands
  - **Load Balancing**: Distributes traffic evenly across containers to optimize performance.
  - **Resilience**: Automatically restarts failed containers and ensures high availability
  - **Health Monitoring**: Tracks the status of containers and underlying infrastructure

RECAP

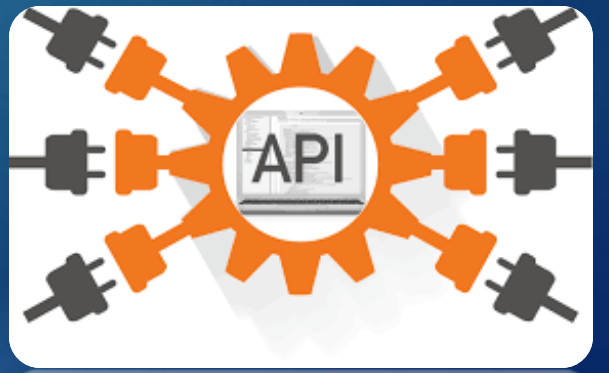
- Kubernetes, often abbreviated as K8s, is the most widely adopted open-source container orchestration platform
- Why Kubernetes?
  - **Automation**: Handles container deployment, scaling, and management automatically
  - **Portability**: Abstracts infrastructure, allowing applications to run across any cloud or on-premise environment
  - **Efficiency**: Optimizes resource utilization through bin-packing and auto-scaling
  - **Extensibility**: Integrates with a wide range of plugins and APIs for advanced functionality



- Amazon Elastic Kubernetes Service (EKS) is a fully managed Kubernetes service that simplifies the deployment, scaling, and management of containerized applications using Kubernetes

RECAP

- APIs are at the core of container orchestration, enabling seamless communication between containers, the orchestration platform, and external systems
- Why APIs Matter in Modern Systems
  - **System Integration**: APIs provide a standardized interface for different systems to communicate, regardless of programming language or platform
  - **Application Communication**: Services and applications within a system interact through APIs to exchange data and perform tasks
  - **Automation and Workflow Orchestration**: APIs enable automated workflows, triggering actions like data processing, system updates, or scaling resources
  - **Extensibility and Modularity**: APIs allow systems to integrate with third-party tools, enhancing functionality without modifying the core architecture
- Let's take a deeper dive into understanding APIs



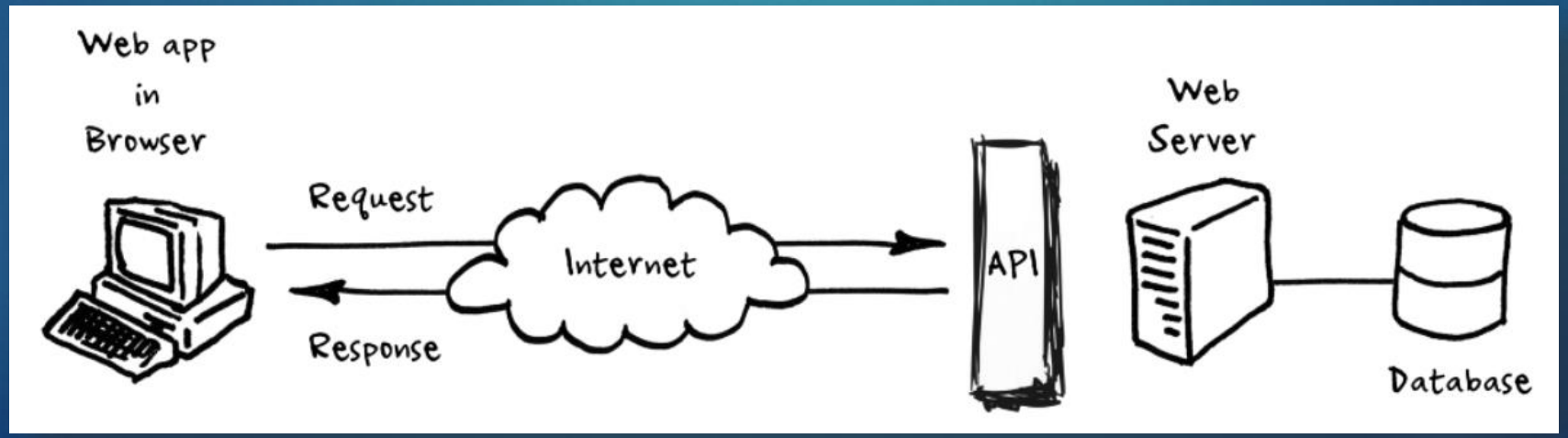
Building APIs in the Cloud

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology

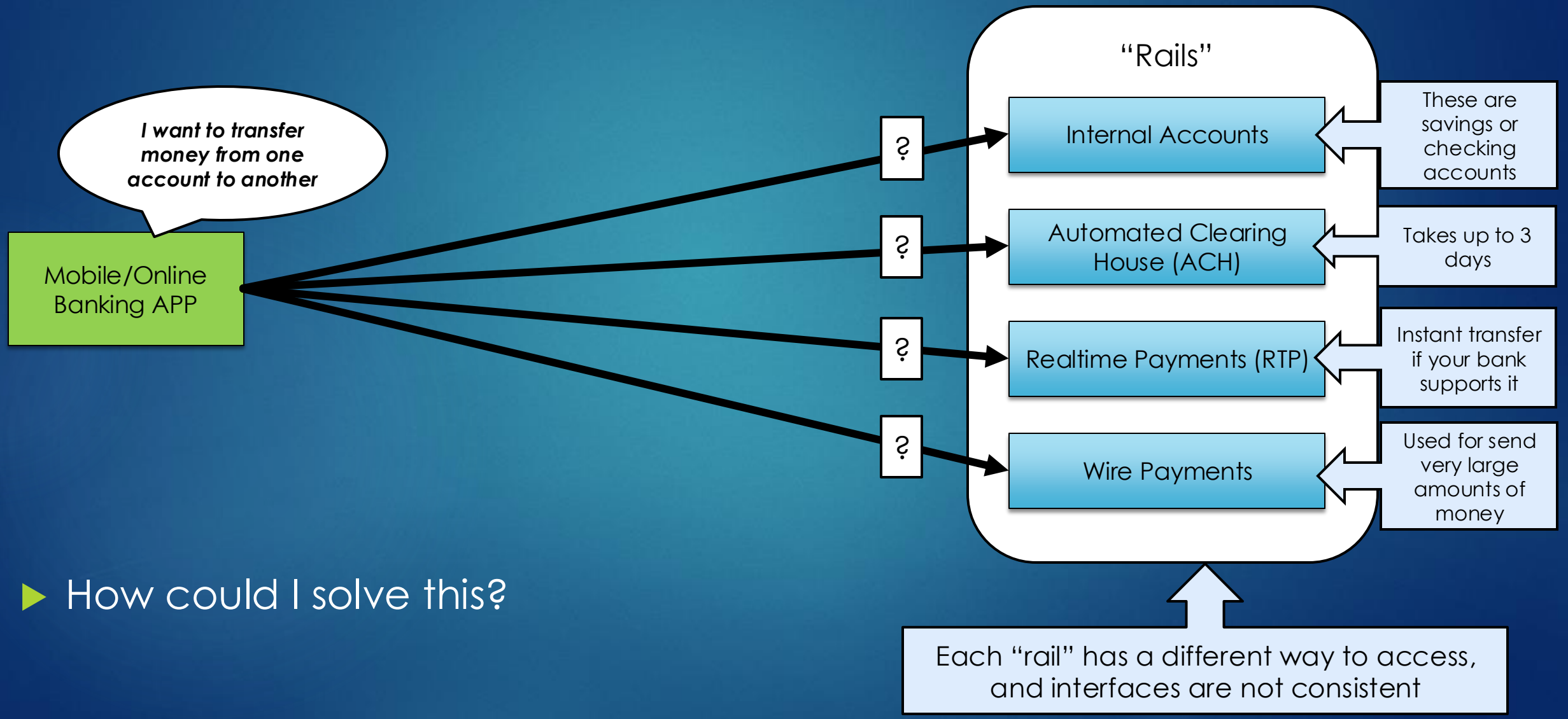
Why are APIs Important?

- APIs (Application Program Interfaces) are vital tools for businesses in all industries
- In basic terms, APIs just allow applications to communicate with one another
- It is not the database or even the server, it is the code that governs the access point(s) for the server



Source: https://medium.com/@perrysetgo/what-exactly-is-an-api-69f36968a41f

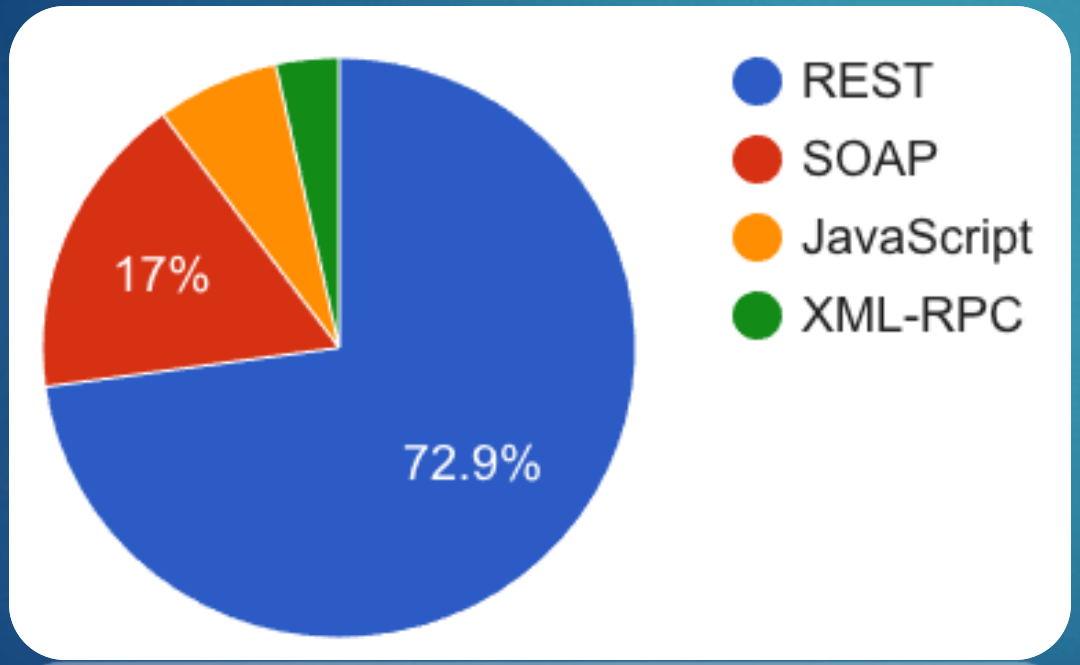
Example: APIs for an Internal System



- How could I solve this?

API Protocols and Data Exchange

- Top API Protocols
  - REST (Representational State Transfer)
  - SOAP (Simple Object Access Protocol)



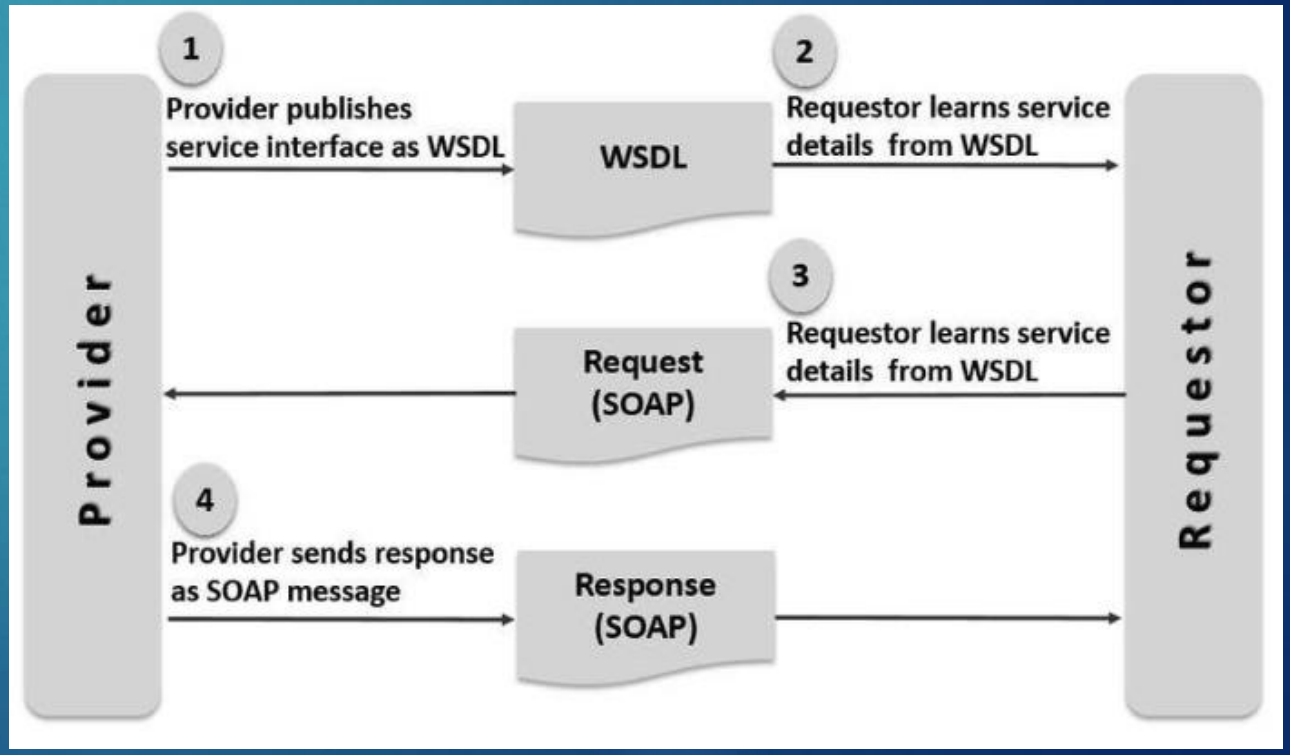
- The most common exchange formats are XML and JSON
  - XML - Extensible Markup Language
    - Standard ways of expressing the structure of the document: XML schema, DTD, etc.
  - JSON (JavaScript Object Notation)
    - Smaller message size
    - More human readable
    - Can be natively loaded as JavaScript object (speed is a value)

Source: https://code.tutsplus.com/articles/the-increasing-importance-of-apis-in-web-development--net-22368

SOAP (Simple Object Access Protocol) Overview

SOAP is a lightweight, XML-based protocol for exchanging information in a decentralized, distributed environment

- Developed in 1998 by several companies including Microsoft and IBM
- Lightweight communication protocol
- Combines a SOAP-based requests and responses with a transport protocol such as HTTP
- Not tied to any programming language
- A Web Services Description Language (WSDL) describes the functionality offered by the web service in XML

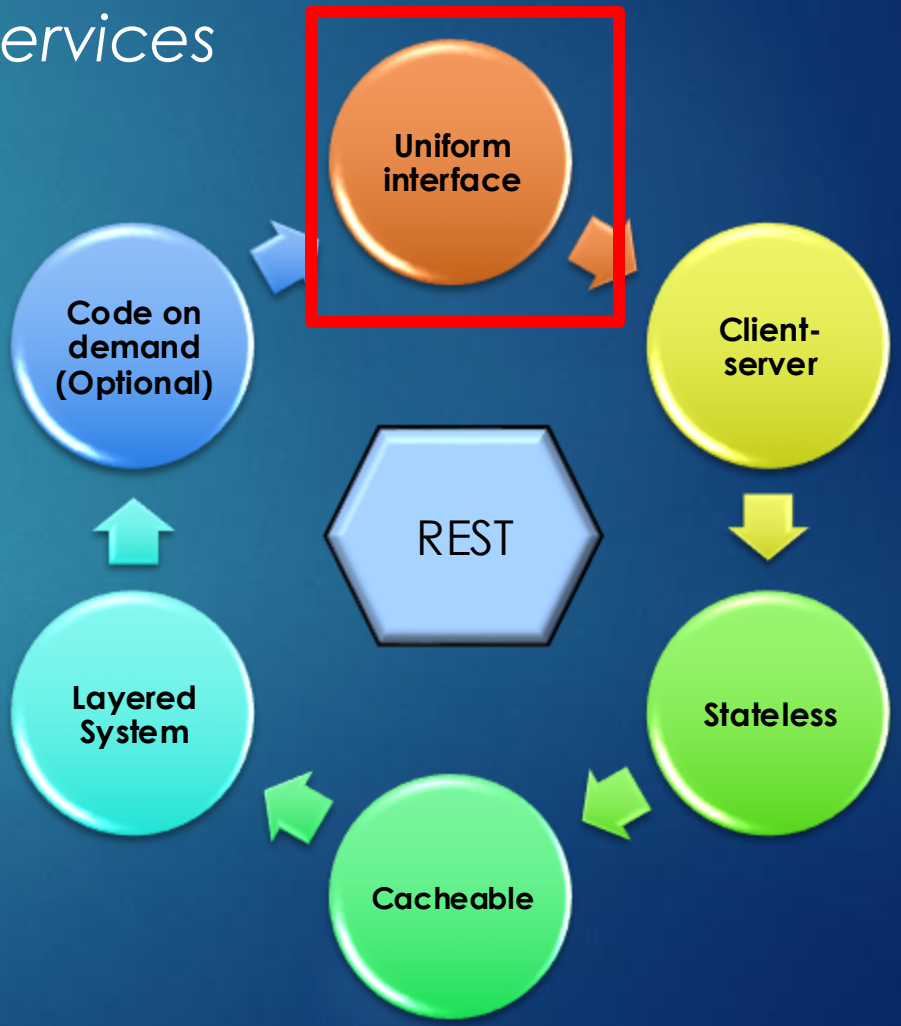


Source: https://www.researchgate.net/figure/SOAP-protocol-SOAP-is-the-master-leader-in-communications-Its-main-purpose-is-to-send_fig1_327054573

REST (Representational State Transfer) Overview

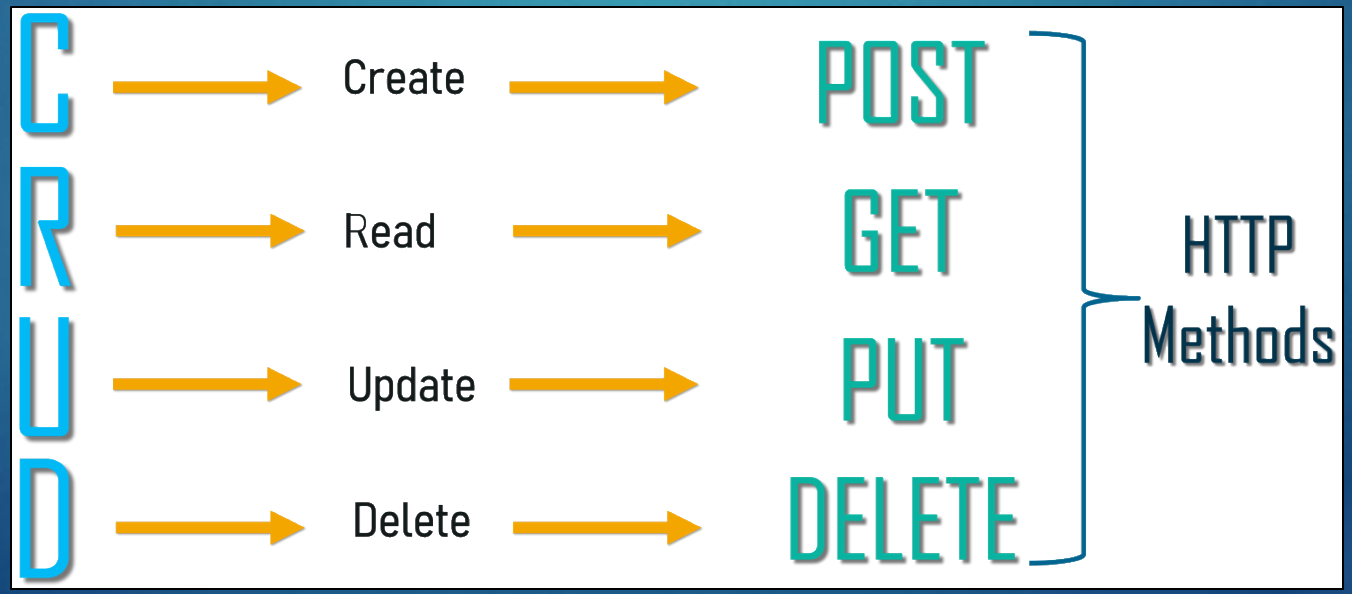
A software architectural style that defines a set of constraints to be used for creating Web services

- REST API defines a set of functions to which the developers can perform requests and receive responses via the HTTP protocol
- It defines the 6 architectural constraints which make any web service a true "RESTful API"
- With the wide usage of HTTP, REST APIs can be used practically by any programming language



Uniform Interface

- You stick to the finite set of operations of the application protocol you're distributing your services upon
- For HTTP, this means that services are restricted to using specific methods (GET, PUT, DELETE and POST)
- This aligns with common storage and database operations (aka "CRUD")

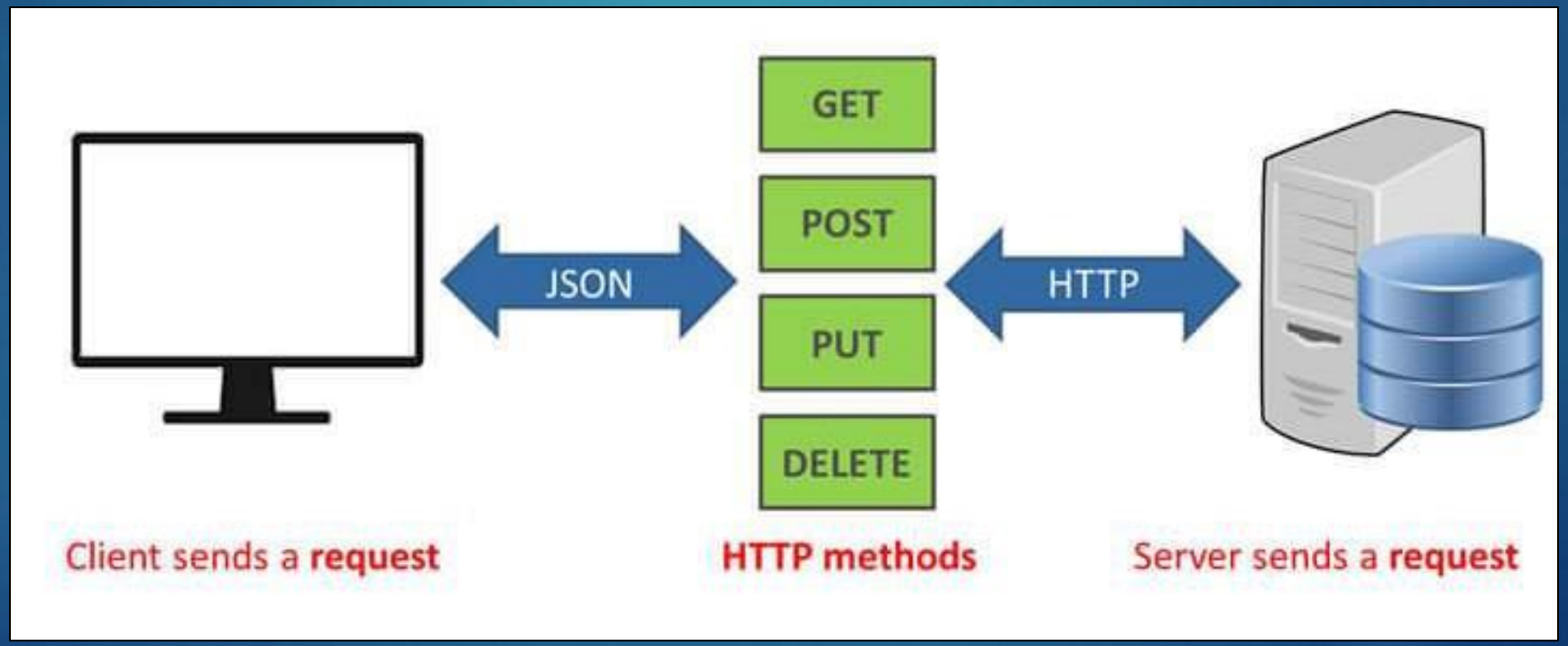


Other Architectural Constraints

- Client-server
  - The client and server much be able to evolve separately without any dependencies on each other. A client should only know about the resource it's requesting
- Stateless
  - Server will not store anything about the latest request. Each and every request is new
  - If client application needs to store stateful information, this should be passed in on each request (e.g. user id)
- Cacheable
  - Caching provides performance improvement as the same resource does not have to be retrieved (e.g. HTML page) improving scalability and performance
- Layered System
  - A client cannot ordinarily tell whether it is connected directly to the end server, or to an intermediary along the way

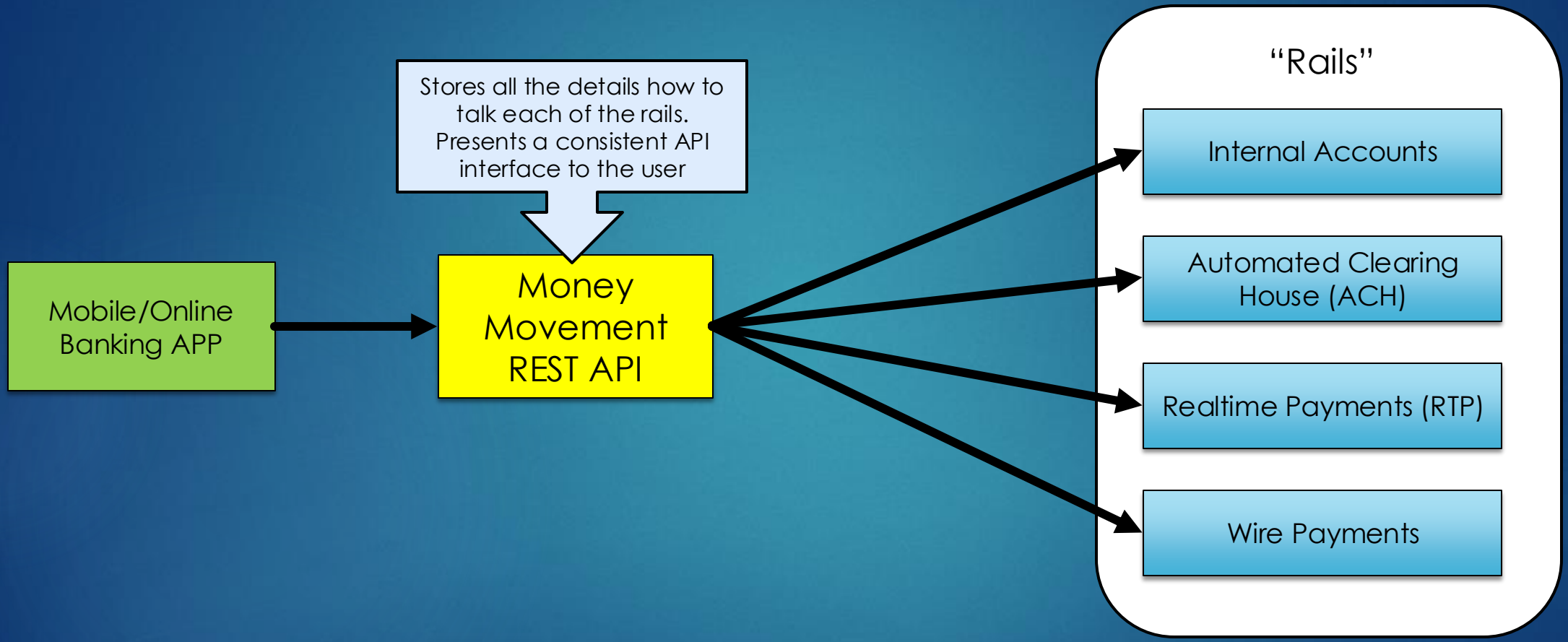
REST API Summary

- We can now define a RESTful API as one that is stateless, it separates concerns between client-server, it allows caching of data client-side and it utilizes standardized base URLs and methods to perform the actions required to manipulate, add or delete data



Source: https://itnext.io/javascript-fundamentals-an-introduction-to-rest-apis-7cbe8a809d3b

APIs for an Internal System



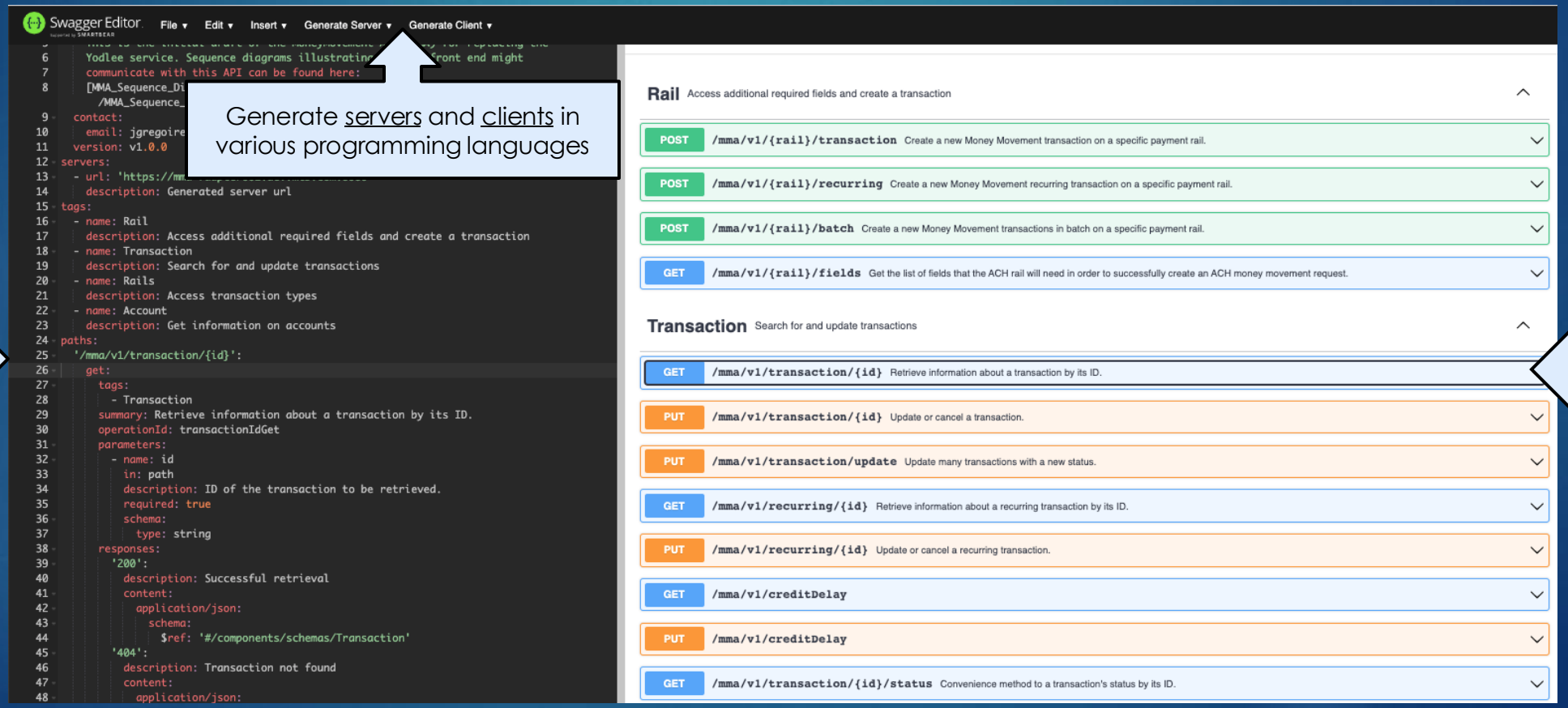
- The Money Movement API is our "contract" with the Mobile/Online Banking
- How do we make sure our contract is correct before we start development?

Open API Specification

- Previously known as the “Swagger Specification”, OpenAPI (version 3.x) is a specification for machine-readable interface files for describing, producing, consuming, and visualizing RESTful web services
- It is language-agnostic
- Why important?
  - With OpenAPI's declarative resource specification, clients can understand and consume services without knowledge of server implementation or access to the server code
  - Applications implemented based on OpenAPI interface files can automatically generate documentation of methods, parameters and models
  - This helps keep the documentation, client libraries, and source code in sync

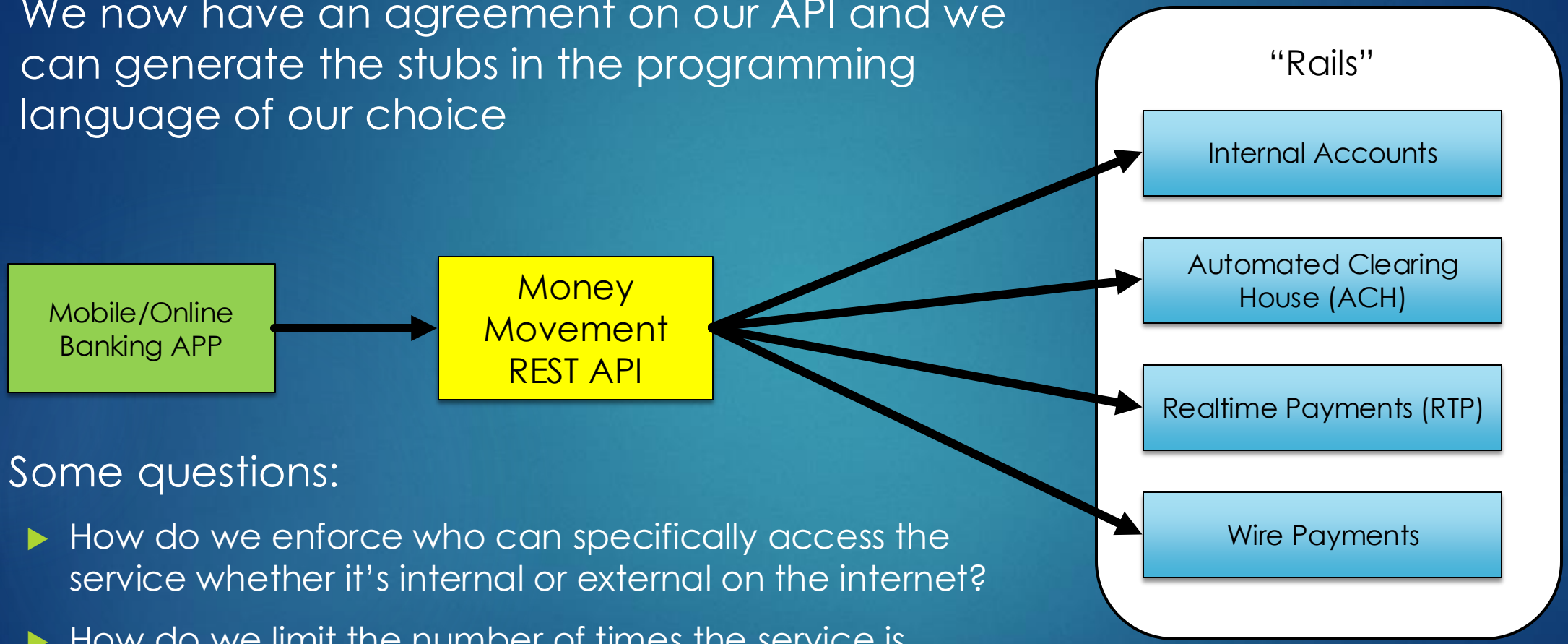
Swagger Tools – Swagger Editor

- Swagger Editor lets you edit OpenAPI specifications in YAML inside your browser and to preview documentations in real time



Back to our Example

- We now have an agreement on our API and we can generate the stubs in the programming language of our choice

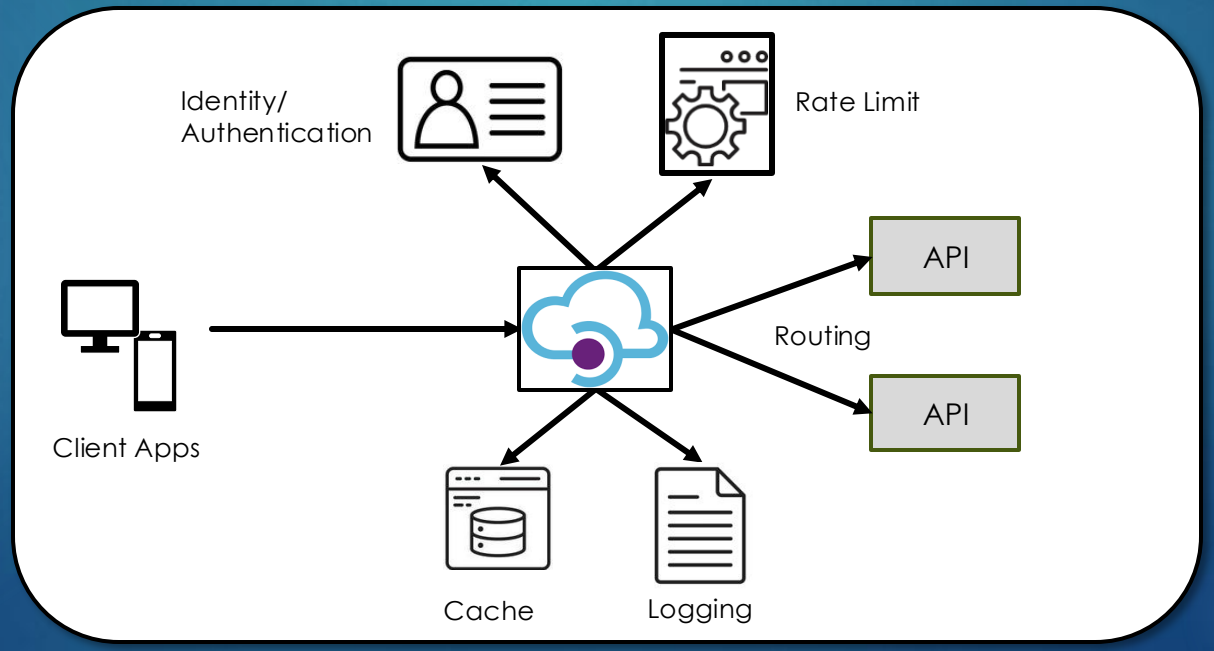


- Some questions:
  - How do we enforce who can specifically access the service whether it's internal or external on the internet?
  - How do we limit the number of times the service is accessed?
  - How can we have a single API call make multiple backend calls?

- Answer: We need an API Gateway

API Gateway

- An API Gateway is a server that is the single-entry point into the system
- It sits between clients and services and acts as a reverse proxy, routing requests from clients to service
- It encapsulates the internal system architecture and provides an API that is tailored to the client

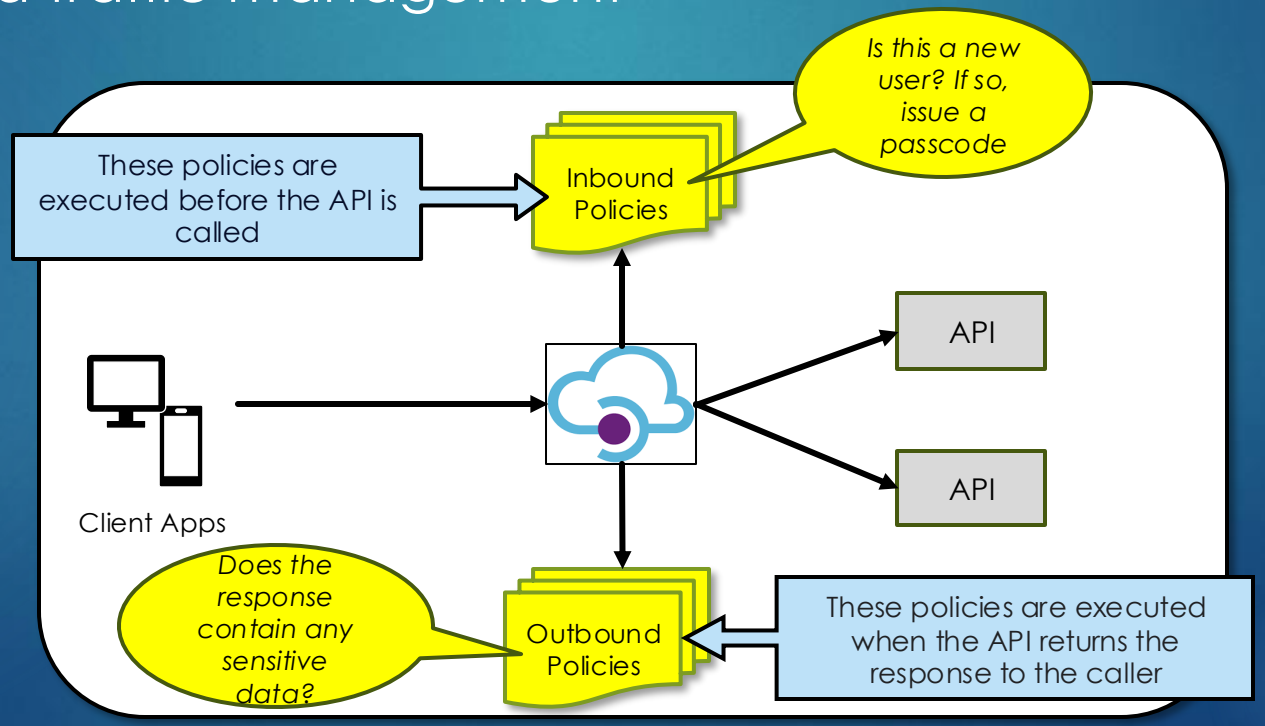


API Gateway Features

- Authentication
  - If the client needs to access data from multiple services, they only need to authenticate once at the gateway reducing latency and ensures authentication processes are consistent across the application
- Input Validation
  - This means ensuring that the client's request contains all the necessary information to complete the request in the correct format before it reaches the service
- Metrics Collection
  - It's the ideal place to collect analytics such as measure how many requests a user is making or how many requests are being relayed to a particular service
  - Allows for "rate limiting" so if a user is sending too many requests, the gateway can reject them
- Response Transformation
  - A mobile devices might need less data than desktop devices, while internal clients might need more information than external clients. An API gateway can be used to account for this, effectively presenting a unique API to each client type

API Gateway Policy

- An API gateway policy is a rule that an API gateway enforces when processing incoming or outgoing requests
- Some of the features previously mentioned may be enforced via policies
- Policies are typically used in four specific areas: authentication, authorization, security, and traffic management



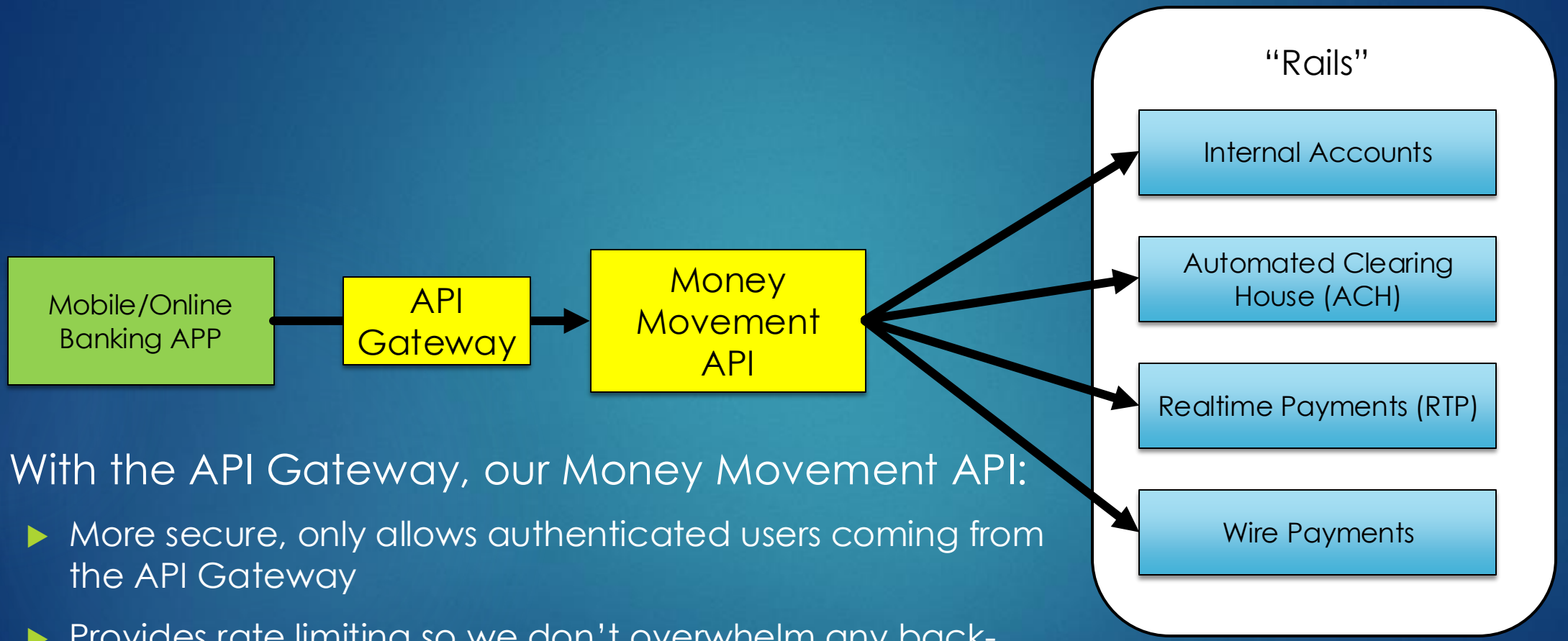
API Gateway Policy - Example

- Policies are generally JSON documents that you can attach to an API to control access

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": [
        "execute-api:/*"
      ]
    },
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": [
        "execute-api:/*"
      ],
      "Condition" : {
        "IpAddress": {
          "aws:SourceIp": ["192.0.2.0/24", "198.51.100.0/24" ]
        }
      }
    }
  ]
}
```

- The following example resource policy denies (blocks) incoming traffic to an API from two specified source IP address blocks

Back to our Example



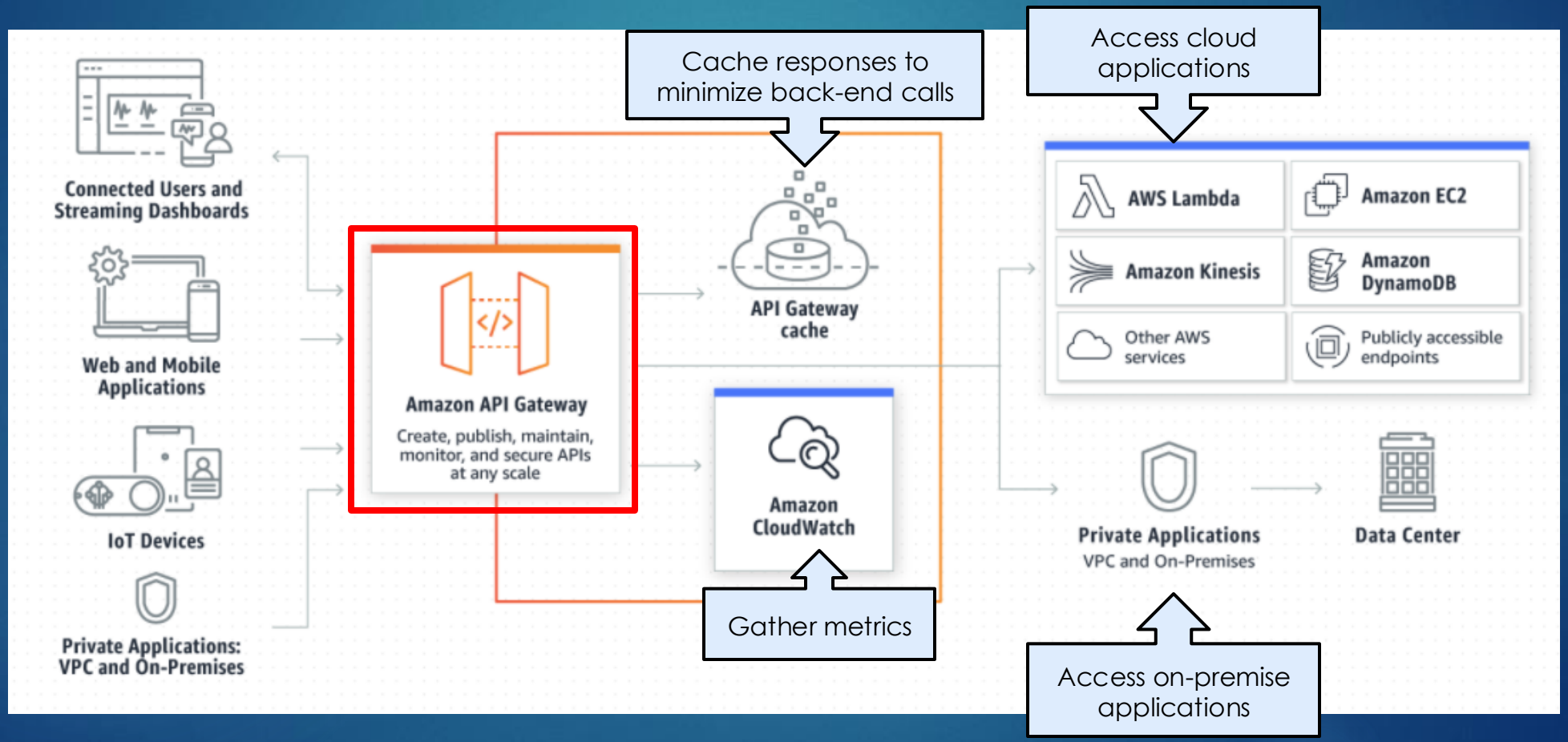
- With the API Gateway, our Money Movement API:
  - More secure, only allows authenticated users coming from the API Gateway
  - Provides rate limiting so we don't overwhelm any back-end rails
  - Combines multiple service calls into a single call, simplifying the API

Amazon API Gateway

- Amazon's API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale
- You can create REST APIs that act as a "front door" for applications to access data, business logic, or functionality from your backend services, such as workloads running on EC2, a web application, or even microservices
- It handles all the tasks involved in accepting and processing up to hundreds of thousands of concurrent API calls, including traffic management, authorization and access control, monitoring, and API version management

Amazon API Gateway

- How it works



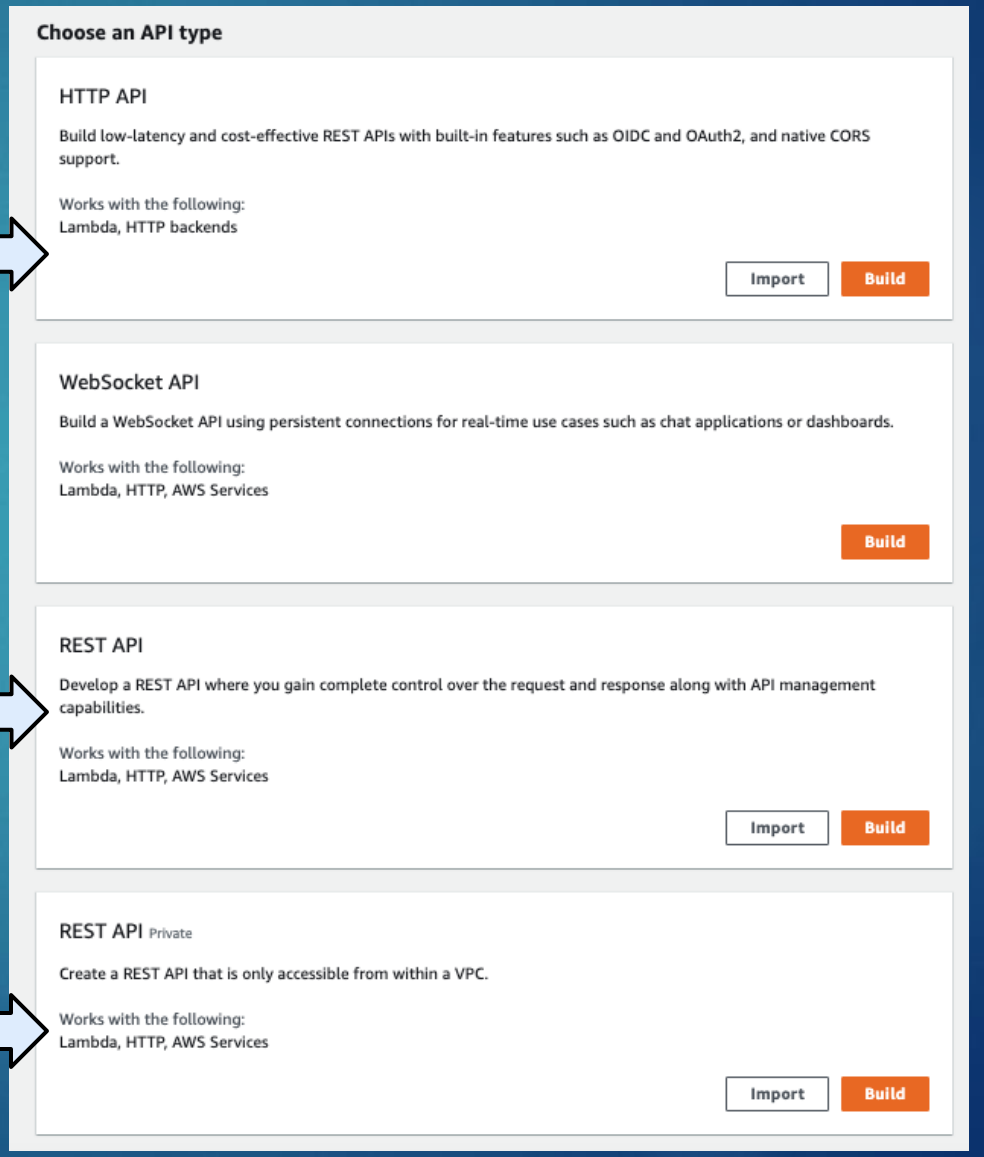
Amazon API Gateway

- Choosing the right API type

Another option for your project that offers public/private APIs

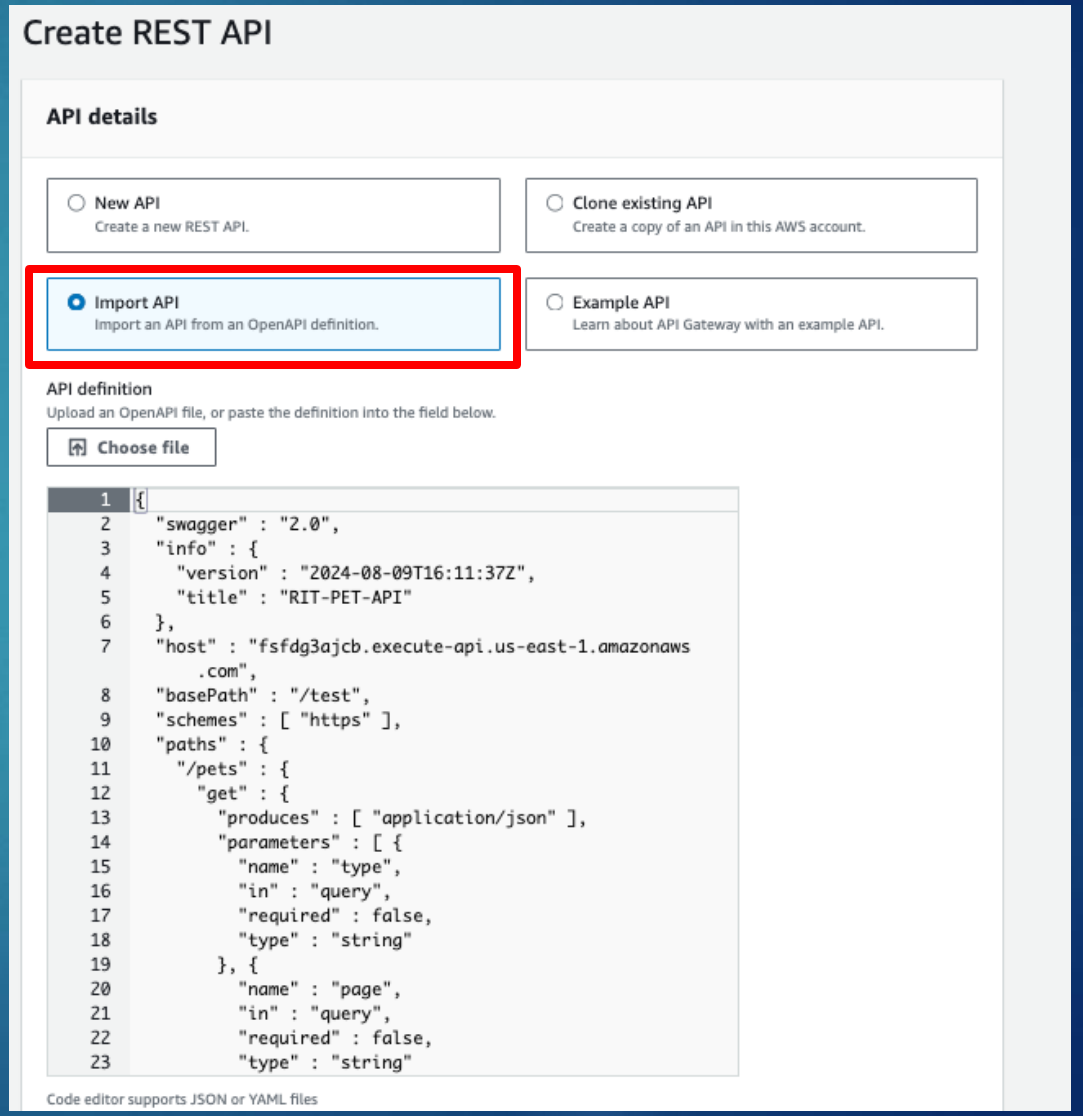
Allows you to create public REST APIs. We will use this for class activities

Choose this for your project if your APIs are internal to your application and they don't need to be public



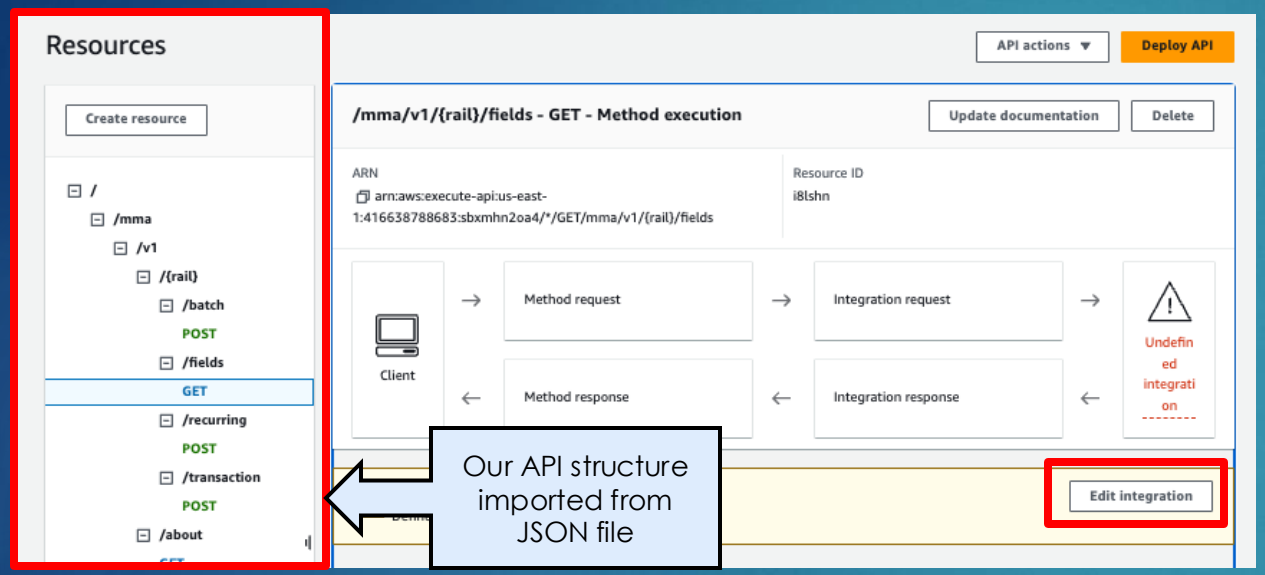
Amazon API Gateway

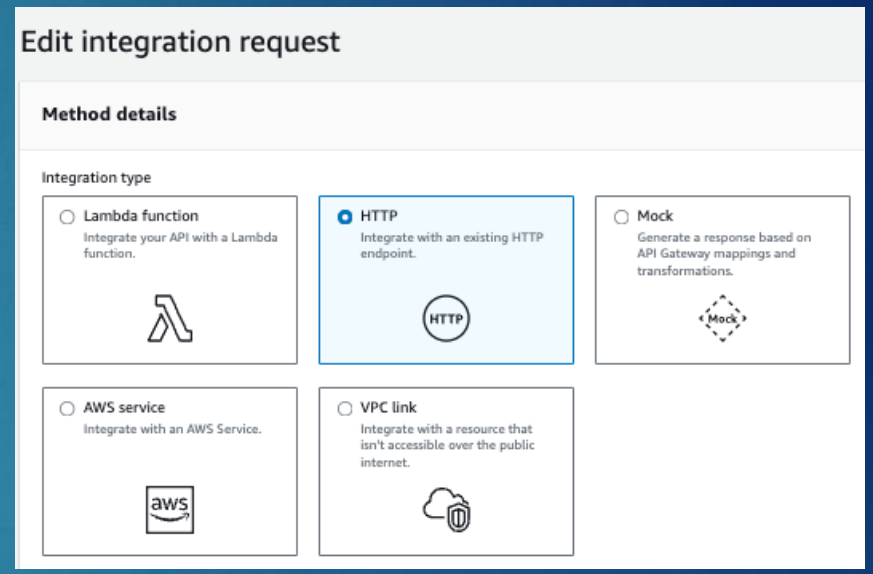
- We can import our Swagger/OpenAPI JSON file directly to API Gateway



Amazon API Gateway

- We can then connect the API Gateway to our back-end services via HTTP or other integrations





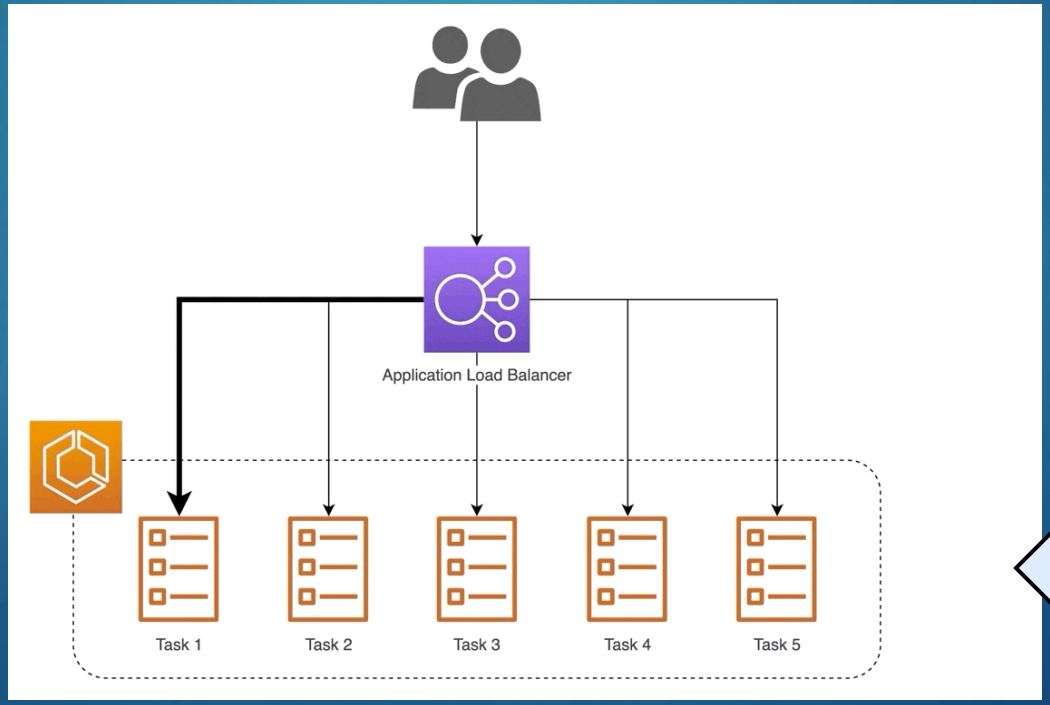
- Final question:
  - If my back-end services are running on multiple servers, how does the API Gateway handle this?
- Answer: Load Balancer

API Gateway + Load Balancer

- An API Gateway would route traffic to a Load Balancer
- The Load Balancer distributes incoming network traffic across two or more servers using algorithms to achieve the best performance
- Some common examples are “Round Robin” and “Least Outstanding Requests”

Load Balancer - Round Robin

- A Round Robin algorithm distributes traffic basically evenly
- It cycles through your targets in order, so each target should receive an equal share of requests
- Due to its simplicity, all load balancers support the round-robin algorithm

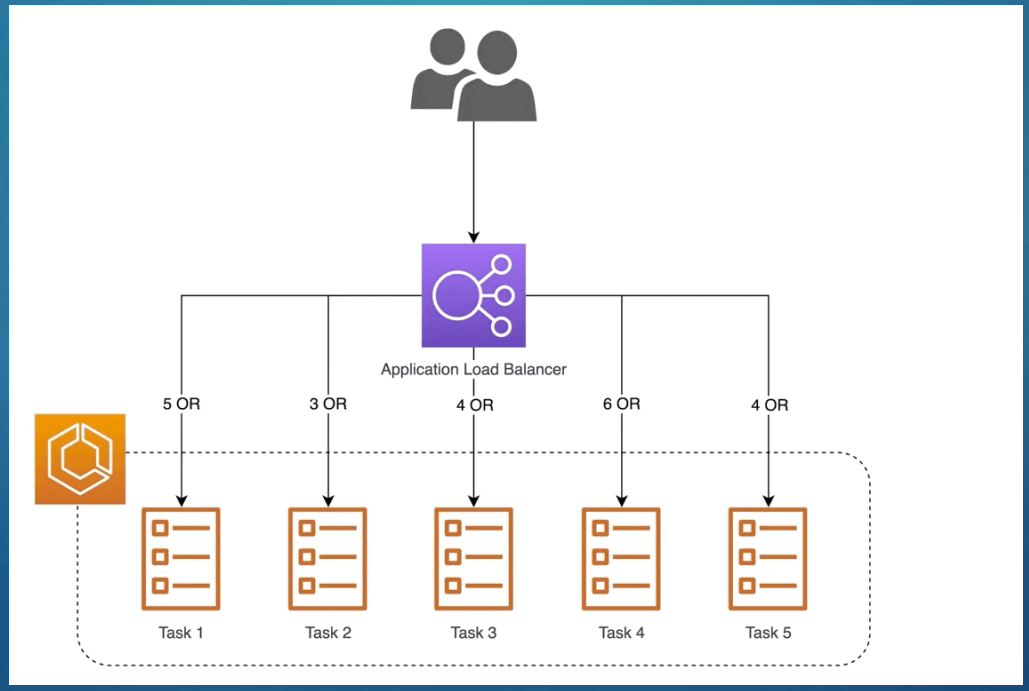


Load Balancing + Auto-scaling = Better Resiliency

Source: https://medium.com/dazn-tech/aws-application-load-balancer-algorithms-765be2eca158

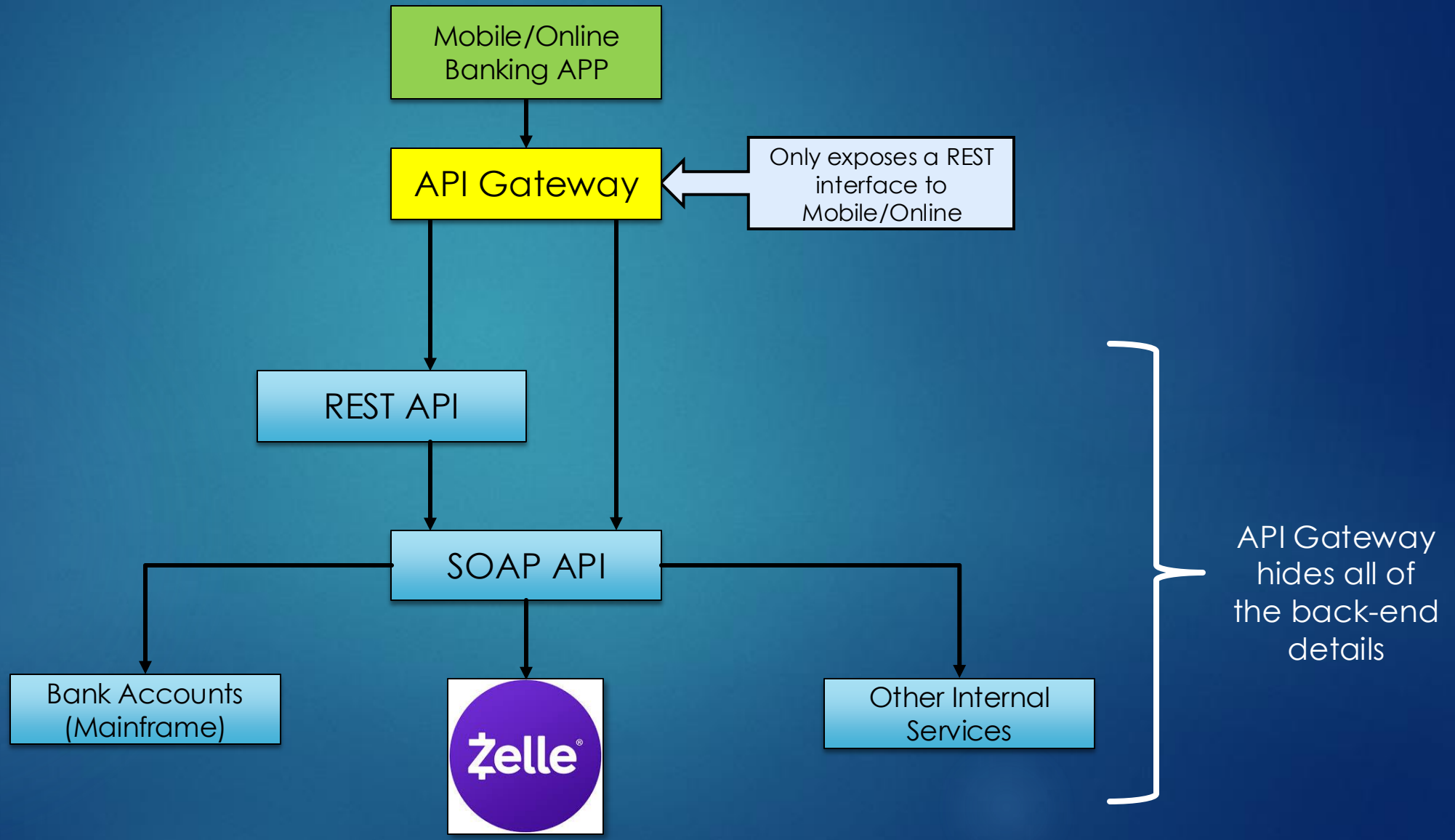
Load Balancer - Least Outstanding Requests

- One major drawback of Round Robin is the assumption that all workloads are the same
  - If they are not, this will add bottlenecks and impact performance
- A more efficient algorithm is Least Outstanding Requests (LOR), which will select the target with the lowest number of requests waiting for responses



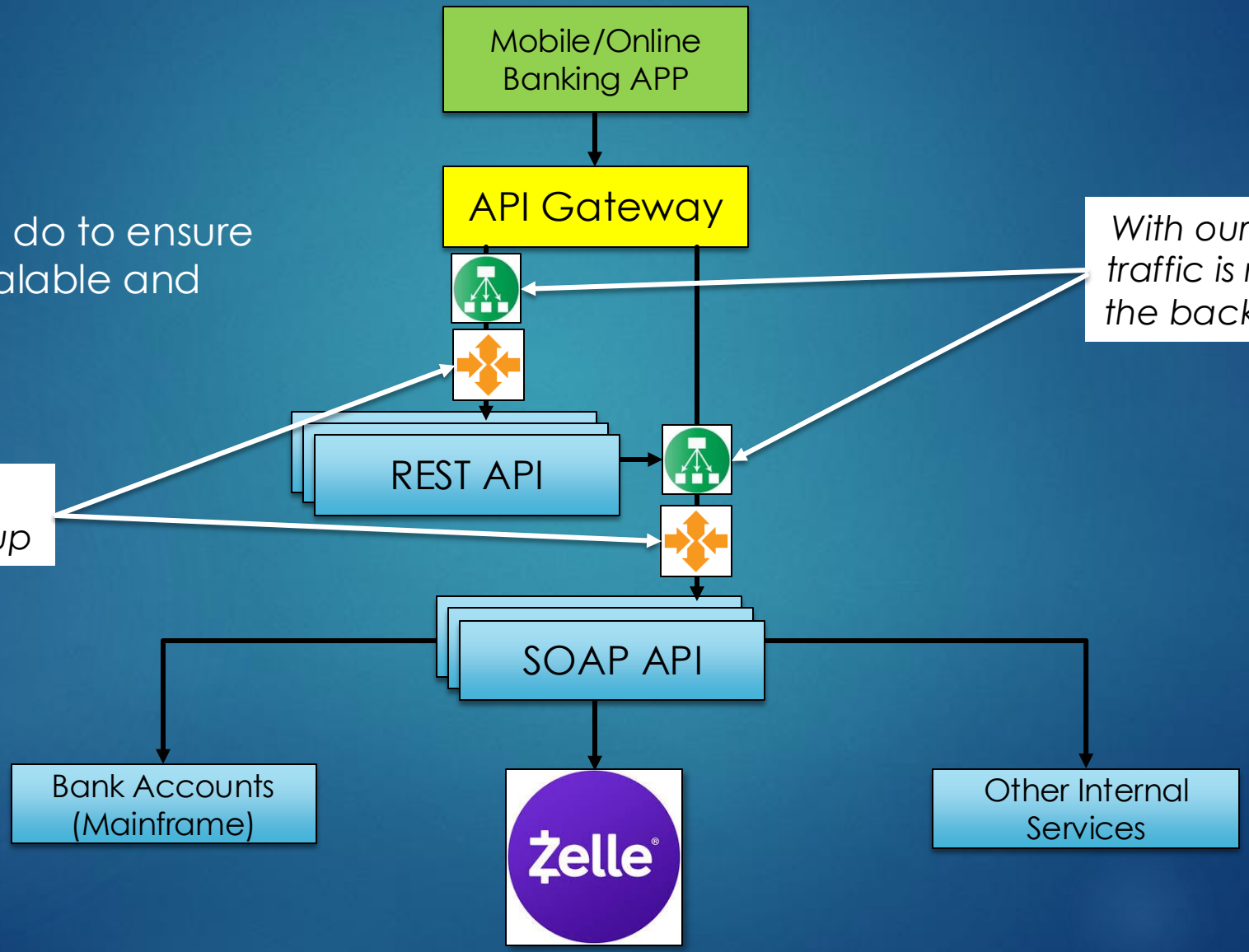
Source: https://medium.com/dazn-tech/aws-application-load-balancer-algorithms-765be2eca158

Example #2: Zelle



Example #2: Zelle

- What else could I do to ensure this solutions is scalable and fault tolerant?



REST APIs and AWS API Gateway

- In this activity, you will be creating a simple REST services and deploying to the AWS API Gateway
- Next, you will import to Swagger to generate the client code (in Python) to connect to your API Gateway
  - Do the activity in Assignments > Activity – Create REST API with Gateway
    - Worth **2 points**
    - There are 3 deliverables for this activity